

Algorithmes et Systèmes Distribués

*Fiche de synthèse — Modèles synchrone et asynchrone,
avec un focus sur les anneaux synchrones et asynchrones*

Ce document reprend et développe les notions du cours M1 ILIA (E. Fabiani, UBO) : modélisation d'un réseau de processus, principe du modèle synchrone, algorithme d'élection de leader sur un anneau synchrone (avec trace complète d'exécution), parcours BFS synchrone, puis le modèle asynchrone et son adaptation sur un anneau.

1. Rappels : modéliser un réseau de processus

1.1 Le réseau comme graphe orienté

Un réseau de processus est modélisé par un **graphe orienté** : les nœuds sont les processus, les arcs sont des **canaux de communication unidirectionnels**. Un processus P1 est *voisin* d'un processus P2 s'il existe un canal direct entre les deux, quel que soit son sens (voisin *entrant* si le canal va de P2 vers P1, *sortant* dans l'autre sens).

1.2 Distance et diamètre

La **distance**(i,j) est la longueur du plus court chemin de i vers j, en nombre de sauts. Le **diamètre** du réseau est la distance maximale prise sur toutes les paires (i,j). Il borne le nombre d'échanges nécessaires pour qu'une information atteigne n'importe quel processus, et sert donc à prévoir la durée d'une diffusion ou la terminaison d'un algorithme.

Exemple : dans un anneau unidirectionnel de 6 processus, le diamètre vaut 5 : dans le pire cas, un message doit traverser les 5 autres processus avant de revenir à son point de départ.

2. Le modèle synchrone

2.1 Pourquoi un modèle synchrone ?

Dans le cas général, les canaux de communication ont des défauts : pertes de messages, délais variables (le récepteur ne peut pas distinguer un message en retard d'un message perdu). Si l'algorithme est itératif — ce qui est généralement le cas — ces défauts entraînent une **décorrélation des états des processus** : ils perdent la notion d'avancement commun.

Le modèle synchrone résout ce problème en **synchronisant les itérations via les communications**, qui deviennent bloquantes *logiciellement* : aucun processus n'a jamais plus d'un tour d'avance sur les autres, et les communications deviennent déterministes — un processus sait qu'il doit envoyer un message et qu'il en recevra un de chacun de ses voisins. Par construction, les interblocages de type *deadlock* sont donc impossibles : tout envoi aura obligatoirement une réception.

Attention à ne pas confondre : le comportement *interne* de chaque processus reste asynchrone (il exécute son code à son propre rythme entre deux points de synchronisation), et les canaux eux-mêmes peuvent techniquement avoir une capacité (FIFO) — ce qui compte, c'est qu'il n'y ait jamais plus d'un message utile en transit par canal à un instant donné.

2.2 Définition formelle

Une itération (un **tour**) est bloquée tant que : (1) un message n'a pas été reçu de chacun des voisins entrants, et (2) un message n'a pas été envoyé à chacun des voisins sortants. Cela constitue un **temps logique global** : le numéro de tour joue le rôle d'horloge commune, et les décalages entre processus sont au plus égaux à 1.

Chaque processus est vu comme un automate à états, défini par : **in(i)/out(i)** (entrées et sorties), **states(i)** (l'ensemble de ses états possibles), **start(i)** (état initial), **trans(i)** (fonction de transition : état courant + messages reçus → nouvel état), et **msgs(i)** (fonction de génération de messages : état courant → message à envoyer sur chaque sortie, éventuellement *null*).

2.3 Le déroulement par tours

- **Étape 1 — Envoi/réception** : chaque processus envoie un message à tous ses voisins sortants et reçoit un message de tous ses voisins entrants (les deux se font en parallèle pour éviter tout interblocage).

- **Étape 2 — Calcul** : chaque processus met à jour son état courant en fonction des messages reçus.
- **Étape 3 — Préparation** : chaque processus prépare les messages à envoyer au tour suivant.

2.4 Le message *null*

Comme un processus doit **toujours** envoyer un message à chaque tour (même s'il n'a rien d'utile à dire), on introduit un message **null** : il symbolise l'absence de message utile, et un message null reçu doit être ignoré. En pratique, il peut être implémenté par un *timeout* : si aucun message n'arrive dans un délai donné, on considère qu'un message null a été reçu. Le message null sert aussi, en simulation, à modéliser des pannes (transformer artificiellement un message utile en null revient à simuler sa perte).

3. Focus : l'anneau synchrone — élection de leader

3.1 Le problème

On considère un **anneau unidirectionnel** : chaque nœud a un identifiant unique, un seul voisin entrant et un seul voisin sortant. Le nombre total de processus n'est pas forcément connu. L'objectif : qu'un seul processus s'élise *leader*, en s'appuyant uniquement sur des communications locales (chaque nœud ne parle qu'à son voisin direct).

3.2 Algorithme (à chaque tour de boucle)

Initialement, chaque processus définit : $id :=$ son identifiant, $message\ sortant := id$, $état :=$ non-leader. Puis, à chaque tour :

envoi(message sortant) et réception(message entrant) ; soit $v =$ message entrant reçu :

- si $v == null \rightarrow$ message sortant $:= null$ (rien à propager)
- si $v > id \rightarrow$ message sortant $:= v$ (l'identifiant reçu est plus grand : on le relaie)
- si $v < id \rightarrow$ message sortant $:= null$ (l'identifiant reçu est plus petit : on l'élimine)
- si $v == id \rightarrow$ état $:= leader$ (l'identifiant a fait le tour complet et revient : c'est le sien)

Idée clé : chaque processus ne relaie que les identifiants plus grands que le sien et élimine les plus petits. Seul le plus grand identifiant de l'anneau survit à chaque étape et finit par parcourir tout l'anneau sans être arrêté, jusqu'à revenir à son émetteur d'origine, qui se déclare alors leader. L'algorithme se termine en **N+1 tours**, où N est le diamètre de l'anneau (N = nombre de processus - 1 pour un anneau).

3.3 Exemple détaillé pas à pas

Anneau de 6 processus, dans le sens $6 \rightarrow 3 \rightarrow 2 \rightarrow 5 \rightarrow 4 \rightarrow 1 \rightarrow$ (retour à 6). Les identifiants initiaux sont ceux des processus eux-mêmes (6, 3, 2, 5, 4, 1). Le tableau ci-dessous donne, pour chaque tour, la valeur portée par chaque canal sortant (message envoyé au tour suivant) :

Canal	Tour 1	Tour 2	Tour 3	Tour 4	Tour 5	Tour 6
6 $\rightarrow$ 3	6	null	null	null	null	null
3 $\rightarrow$ 2	3	6	null	null	null	null
2 $\rightarrow$ 5	2	3	6	null	null	null
5 $\rightarrow$ 4	5	null	null	6	null	null
4 $\rightarrow$ 1	4	5	null	null	6	null
1 $\rightarrow$ 6	1	4	5	null	null	6

Lecture : au tour 1, chaque nœud envoie simplement son propre identifiant. Au tour 2, par exemple, le nœud 3 a reçu 6 ($>$ à son id 3) et le relaie ; le nœud 2 a reçu 3 ($>$ à son id 2) et le relaie ; mais le nœud 6 a reçu 1 ($<$ à son id 6) et envoie *null*. On observe que seul l'identifiant 6, le plus grand, continue de se propager sans jamais être arrêté (les autres identifiants sont progressivement éliminés dès qu'ils rencontrent un nœud d'id supérieur). Au tour 6, le nœud 6 reçoit son propre identifiant ($6 == 6$) : il devient **leader**.

6 processus, diamètre = 5, et l'algorithme se termine en 6 tours = N+1, conformément à la règle générale.

3.4 Utilité et variantes

- La structure d'anneau peut être une topologie **logique** plaquée sur un réseau quelconque.
- Un second tour complet (N+1 tours) permet au leader de **se faire connaître** de tous et de provoquer la terminaison des autres processus, utile si le diamètre est inconnu au départ.

- Le leader élu peut ensuite **initier des calculs globaux** (max, sommes, calculs booléens, détection de terminaison...) dont il récolte et rediffuse les résultats.
- Si les nœuds comptent les tours, le leader connaît le diamètre exact de l'anneau et peut le retransmettre à tous.

4. Complément : le parcours BFS synchrone

Sur une topologie **quelconque** (pas nécessairement un anneau), le parcours en largeur (*Breadth First Search*) construit un arbre de recouvrement à partir d'un nœud source *root*, en supposant le réseau fortement connecté (distance finie entre toute paire de nœuds) et le diamètre inconnu.

Chaque processus a un état booléen *visité* (VRAI seulement pour *root* au départ). À chaque tour : si *visité* = FAUX et qu'un message *search* est reçu, alors le processus passe à *visité* = VRAI, retient l'un des canaux d'où provient *search* comme étant celui de son **père** dans l'arbre, et relaie *search* à ses propres voisins sortants au tour suivant. Sinon, il envoie *null*. L'algorithme se termine en N tours, où N est le diamètre du réseau.

Utilité : en propageant d'autres valeurs le long de ce même parcours, on peut diffuser une information à tous les processus ; avec des canaux bidirectionnels, chaque fils peut répondre à son père (remontée de résultats). Combiné à un identifiant par processus, un BFS lancé simultanément par chacun permet aussi d'élire un leader (celui qui obtient le maximum des identifiants) et de calculer le diamètre du réseau (maximum des profondeurs des arbres).

5. Le modèle asynchrone

5.1 Motivations

Le modèle synchrone vide les canaux à chaque tour (visibilité locale immédiate des voisins), ce qui fiabilise l'écriture des algorithmes mais impose des communications fréquentes : c'est le processus le plus lent qui fixe la cadence de tout le système. Le modèle asynchrone est moins contraint et plus proche des communications réseau réelles : les processus émettent librement, les canaux ont une capacité de stockage, les émissions/réceptions sont gérées par des automates indépendants du processus applicatif, et il n'y a pas de limite sur le temps de transmission.

Exemple concret : un câble transocéanique en fibre optique de 5000 km à 2,5 Gb/s peut contenir plus de 3000 paquets en transit simultanément — le medium physique lui-même agit comme un tampon, ce que le modèle synchrone (contenance nulle) ne peut pas représenter fidèlement.

5.2 Le canal asynchrone

On introduit un **canal asynchrone** $C(i,j)$ reliant deux processus, avec deux opérations totalement **découplées** : $send(i)(m)$ (l'émetteur dépose le message et repart aussitôt) et $receive(j)(m)$ (le récepteur consulte le canal à son propre rythme). Contrairement au mode synchrone, l'émetteur n'attend pas que le récepteur ait traité le message.

En pratique, un canal est modélisé par un processus qui encapsule une **FIFO** (file premier-entré premier-sorti) : un indice *write* pour la prochaine écriture, un indice *read* pour la prochaine lecture. L'opération *receive* se décompose en une requête de lecture (envoyée au canal) suivie de la donnée effectivement renvoyée. La capacité du canal, les pertes ou le changement d'ordre des messages peuvent être simulés respectivement par la taille de la file, des effacements ou des permutations internes.

6. Focus : l'anneau asynchrone — élection de leader

On reprend le même problème que pour l'anneau synchrone (assembler des processus par des canaux directionnels, chacun avec un identifiant unique), mais avec des canaux **asynchrones**.

Algorithme :

- Chaque processus émet son identifiant *id* sur son canal sortant.

- À la réception d'un identifiant *rcvID*, il le compare au sien : si $rcvID > id$, il ré-émet *rcvID* ; si $rcvID < id$, il ne fait rien ; si $rcvID = id$, il devient *leader*.

6.1 Ce qui change par rapport à la version synchrone

- Il n'y a plus de **message null** : un processus n'émet que lorsqu'il a effectivement un identifiant à relayer, il n'y a pas d'obligation d'émettre à chaque « tour » puisque la notion de tour global n'existe plus.
- Les messages peuvent **s'accumuler dans les nœuds en nombre arbitraire** : un processus lent à consommer ses messages entrants peut voir sa file grossir, alors qu'en synchrone chaque canal ne contient jamais plus d'un message.
- Il n'y a plus de notion de temps logique global commun (pas de numéro de tour partagé) : chaque processus avance à son propre rythme, ce qui complique le raisonnement sur la terminaison mais colle davantage à la réalité d'un réseau physique.
- La correction de l'algorithme (un seul leader élu, et cela se termine) doit alors être prouvée sans pouvoir s'appuyer sur un comptage de tours synchronisé.

7. Synchrone vs asynchrone : tableau comparatif

	Modèle synchrone	Modèle asynchrone
Rythme des tours	Global, commun à tous les processus (décalage max. de 1 tour)	Aucun rythme global : chaque processus avance à son propre rythme
Contenance des canaux	Au plus 1 message utile en transit	Capacité arbitraire (FIFO), les messages peuvent s'accumuler
Émission	Obligatoire à chaque tour (message null si rien à dire)	Libre : un processus émet quand il veut, autant qu'il veut
send / receive	Couplés (bloquants ensemble)	Totalement découplés
Interblocages (deadlock)	Impossibles par construction	Doivent être évités par la conception
Terminaison de l'élection sur un anneau	En N+1 tours exactement (N = diamètre)	Pas de borne en "tours" ; dépend des délais réels de transmission
Proximité du réel	Modèle de conception, plus simple à prouver correct	Plus proche des réseaux physiques réels

À retenir : le modèle synchrone facilite la conception et la preuve de correction (couplage fort, terminaison prévisible en tours), au prix d'une cadence limitée par le processus le plus lent. Le modèle asynchrone colle davantage à la réalité des réseaux (aucune contrainte de rythme, canaux avec mémoire), mais complique le raisonnement sur la terminaison et exige des précautions explicites contre les interblocages.